

Sprint 22 Structured Notes: Searching, Recursion, Divide and Conquer, and Merge Sort

0. What Sprint 22 Is About

Sprint 22 is about searching and sorting data more intelligently.

Before this sprint, you learned many JavaScript tools:

- values
- variables
- data types
- strings
- numbers
- booleans
- conditions
- functions
- arrays
- loops
- objects
- callbacks
- higher-order array methods
- Sets and Maps
- DOM selection
- DOM updates
- events
- accessibility
- forms
- dates and media APIs
- storage
- debugging
- regex
- classes
- Big O basics
- problem-solving workflow

Sprint 21 taught you to ask:

Is my solution correct, clear, and efficient?

Sprint 22 applies that thinking to searching and sorting.

The main question in Sprint 22 is:

When I need to find or organize data, should I check everything, or can I use a smarter method?

By the end of Sprint 22, you should understand:

- linear search
- binary search
- why binary search needs sorted data
- `low`, `high`, and `mid` in binary search
- time complexity of searching
- recursion
- base cases
- recursive calls
- call stack basics
- work before a recursive call
- work after a recursive call
- divide and conquer
- merging two sorted arrays
- merge sort
- time and space complexity of merge sort
- common beginner mistakes
- how to dry-run search and sort algorithms

Part A: Current Progress Before Sprint 22

1. What You Already Know

Before Sprint 22, you already have enough JavaScript foundation to understand algorithms.

You should already know how to:

Area	What You Should Know
JavaScript basics	Use variables, values, types, operators, and conditions.
Functions	Write reusable logic with parameters and return values.
Arrays	Store ordered data and access items by index.
Loops	Repeat work safely with <code>for</code> , <code>for...of</code> , and <code>while</code> .
Objects	Store structured key-value data.
Debugging	Inspect errors and check values step by step.
Big O basics	Think about how code grows as input gets bigger.

Simple summary:

Earlier sprints taught you how to write JavaScript. Sprint 22 teaches you how to use JavaScript to solve search and sort problems more efficiently.

2. Why Sprint 22 Matters

A lot of real programming problems involve finding or organizing data.

Examples:

- Find a username in a list.
- Find a product by ID.
- Check if a score exists in an array.
- Search a sorted list of numbers.
- Sort tasks by priority.
- Sort products by price.
- Sort names alphabetically.

You could always check every item, but that may become slow when the input gets large.

Sprint 22 teaches better choices.

Part B: The Core Ideas of Sprint 22

3. Main Ideas

Sprint 22 has four main ideas.

Concept	Simple Meaning
Linear search	Check items one by one.
Binary search	Search sorted data by repeatedly cutting the search area in half.
Recursion	A function calls itself until it reaches a stopping condition.
Merge sort	Split an array, sort the pieces, then merge the pieces back together.

These concepts are connected.

Binary search and merge sort both use a strategy called divide and conquer.

Part C: Linear Search

4. What Is Linear Search?

Linear search means:

Start at the beginning of a list and check each item until you find the target.

Example:

```
const numbers = [10, 25, 30, 40];  
const target = 30;
```

JavaScript checks:

```
10 === 30? no  
25 === 30? no  
30 === 30? yes
```

The target `30` is found at index `2`.

5. Why It Is Called Linear Search

It is called linear because the search moves in a straight line:

```
start → item 1 → item 2 → item 3 → item 4 → end
```

It does not skip around.

It checks one item after another.

6. Linear Search Function

```
function linearSearch(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
  
  return -1;  
}
```

```
}  
  
console.log(linearSearch([10, 25, 30, 40], 30)); // 2  
console.log(linearSearch([10, 25, 30, 40], 99)); // -1
```

7. How the Code Works

```
function linearSearch(arr, target) {
```

This creates a function named `linearSearch`.

It receives:

- `arr`: the array to search
- `target`: the value we want to find

```
for (let i = 0; i < arr.length; i++) {
```

This loop starts at index `0` and continues until the end of the array.

```
if (arr[i] === target) {  
  return i;  
}
```

If the current item matches the target, return the index.

```
return -1;
```

If the loop finishes and nothing matched, return `-1`.

8. Why Return `-1`?

JavaScript array indexes start at `0`.

Valid indexes look like this:

```
0, 1, 2, 3, 4, ...
```

So `-1` is not a normal valid index.

That makes `-1` useful because it means:

The target was not found.

This is also similar to how some JavaScript methods behave, such as `indexOf()`.

Example:

```
console.log([10, 20, 30].indexOf(20)); // 1
console.log([10, 20, 30].indexOf(99)); // -1
```

9. Linear Search Dry Run

Code:

```
linearSearch([10, 25, 30, 40], 30);
```

Dry run:

Step	i	arr[i]	target	Match?	What Happens
1	0	10	30	No	Continue
2	1	25	30	No	Continue
3	2	30	30	Yes	Return <code>2</code>

Result:

```
2
```

10. When Linear Search Is Useful

Use linear search when:

- the array is not sorted
- the array is small
- you only need a simple solution
- you cannot make assumptions about the order of data

Example:

```
const names = ["Maya", "Alex", "Sangeeth", "Lina"];
console.log(linearSearch(names, "Sangeeth")); // 2
```

11. Linear Search Time Complexity

Linear search is **$O(n)$** time.

Simple meaning:

If the array gets bigger, the possible number of checks grows at the same rate.

Array Size	Worst-Case Checks
10 items	10 checks
100 items	100 checks
1,000 items	1,000 checks
1,000,000 items	1,000,000 checks

The worst case happens when:

- the target is at the end, or
- the target is not in the array

12. Linear Search Space Complexity

The basic linear search function uses **$O(1)$** extra space.

Why?

It only uses a small number of variables:

- `i`
- `arr`
- `target`

It does not create a new array that grows with the input.

Part D: Binary Search

13. What Is Binary Search?

Binary search is a faster search algorithm for sorted data.

Simple meaning:

Binary search checks the middle item, then removes half of the remaining search area.

Important rule:

Binary search only works correctly if the data is sorted.

Example sorted array:

```
const numbers = [3, 8, 12, 18, 25, 40, 60];
```

Target:

25

Instead of checking from left to right, binary search checks the middle first.

14. Binary Search Mental Model

Think about looking for a word in a dictionary.

You do not start at the first page and read every word.

Instead:

1. You open near the middle.
2. If the word should come later, you ignore the left half.
3. If the word should come earlier, you ignore the right half.
4. You repeat this until you find the word.

That is the idea of binary search.

15. Binary Search Uses Three Indexes

Binary search usually tracks three positions:

Variable	Meaning
<code>low</code>	The first index of the current search area.
<code>high</code>	The last index of the current search area.
<code>mid</code>	The middle index of the current search area.

Example:

```
Array: [3, 8, 12, 18, 25, 40, 60]
Index: 0  1  2  3  4  5  6
```

At the start:

```
let low = 0;
let high = arr.length - 1;
```

So:

```
low = 0
high = 6
```

The middle index is calculated like this:

```
const mid = Math.floor((low + high) / 2);
```

16. Why Use `Math.floor()`?

Sometimes the middle calculation creates a decimal.

Example:

```
(0 + 6) / 2; // 3
```

That is fine.

But another example:

```
(0 + 5) / 2; // 2.5
```

Array indexes cannot be decimals.

So we use `Math.floor()` to round down:

```
Math.floor(2.5); // 2
```

17. Binary Search Dry Run

Array:

```
[3, 8, 12, 18, 25, 40, 60]
```

Target:

```
25
```

Indexes:

```
Index: 0  1  2  3  4  5  6
Value: 3  8 12 18 25 40 60
```

Dry run:

Step	low	high	mid	arr[mid]	Decision
1	0	6	3	18	25 is bigger, so search right half.
2	4	6	5	40	25 is smaller, so search left half.
3	4	4	4	25	Found. Return <code>4</code> .

Result:

```
4
```

18. Binary Search Code

```
function binarySearch(arr, target) {  
  let low = 0;  
  let high = arr.length - 1;  
  
  while (low <= high) {  
    const mid = Math.floor((low + high) / 2);  
  
    if (arr[mid] === target) {  
      return mid;  
    }  
  
    if (arr[mid] < target) {  
      low = mid + 1;  
    } else {  
      high = mid - 1;  
    }  
  }  
  
  return -1;  
}  
  
console.log(binarySearch([3, 8, 12, 18, 25, 40, 60], 25)); // 4  
console.log(binarySearch([3, 8, 12, 18, 25, 40, 60], 99)); // -1
```

19. How the Code Works

```
let low = 0;
```

Start the search at the first index.

```
let high = arr.length - 1;
```

Start the search ending at the last index.

```
while (low <= high) {
```

Keep searching while the search area is still valid.

If `low` becomes greater than `high`, there is no search area left.

```
const mid = Math.floor((low + high) / 2);
```

Find the middle index.

```
if (arr[mid] === target) {  
  return mid;  
}
```

If the middle value is the target, return the middle index.

```
if (arr[mid] < target) {  
  low = mid + 1;  
}
```

If the middle value is too small, ignore the left half.

```
else {  
  high = mid - 1;  
}
```

If the middle value is too large, ignore the right half.

```
return -1;
```

If the loop ends, the target was not found.

20. Why `low <= high`?

The condition is:

```
while (low <= high)
```

This means:

Keep searching while there is at least one item left to check.

If `low` and `high` are equal, there is exactly one item left.

Example:

```
low = 4  
high = 4
```

That means index `4` still needs to be checked.

If you used this instead:

```
while (low < high)
```

You might skip the final item.

Beginner rule:

For standard binary search, use `low <= high`.

21. Why Sorted Input Matters

Binary search depends on this idea:

```
if (arr[mid] < target) {  
    low = mid + 1;  
}
```

This means:

If the middle value is too small, the target must be on the right side.

That is only true when the array is sorted.

Correct use:

```
binarySearch([3, 8, 12, 18, 25, 40, 60], 25);
```

Incorrect use:

```
binarySearch([40, 3, 25, 8, 60], 25);
```

The second array is not sorted.

Binary search may throw away the wrong half and give the wrong answer.

22. Binary Search Time Complexity

Binary search is **$O(\log n)$** time.

Simple meaning:

The search area gets cut in half each time.

Example:

Array Size	Approximate Max Checks
8	3
16	4
32	5
1,024	10
1,048,576	20

This is much faster than linear search for large sorted arrays.

23. Binary Search Space Complexity

The iterative binary search version uses **$O(1)$** extra space.

Why?

It only uses a few variables:

- low
- high
- mid

It does not create a new array.

24. Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Works on unsorted data?	Yes	No
Works on sorted data?	Yes	Yes
Basic idea	Check every item one by one.	Check the middle and remove half.

Feature	Linear Search	Binary Search
Time complexity	$O(n)$	$O(\log n)$
Easier for beginners?	Yes	A little harder
Best use	Small or unsorted data	Large sorted data

Simple rule:

Use linear search when data is unsorted. Use binary search when data is sorted and large enough to benefit from faster searching.

Part E: Recursion

25. What Is Recursion?

Recursion means:

A function calls itself.

Example idea:

```
function sayAgain() {
  sayAgain();
}
```

But this version is dangerous because it never stops.

A recursive function needs a stopping rule.

That stopping rule is called the base case.

26. The Two Required Parts of Recursion

Every useful recursive function needs two things:

Part	Meaning
Base case	The condition that stops recursion.
Recursive case	The part where the function calls itself.

Without a base case, recursion continues until JavaScript runs out of call stack space.

27. What Is a Base Case?

A base case is the stopping condition.

Simple meaning:

The base case tells the function when to stop calling itself.

Example:

```
if (number < 1) {  
  return;  
}
```

This says:

If the number is less than 1, stop.

28. Basic Recursive Countdown

```
function countdown(number) {  
  if (number < 1) {  
    return;  
  }  
  
  console.log(number);  
  countdown(number - 1);  
}  
  
countdown(3);
```

Output:

```
3  
2  
1
```

29. How Countdown Works

```
countdown(3)  
  number is 3  
  print 3
```

```
call countdown(2)

countdown(2)
  number is 2
  print 2
  call countdown(1)

countdown(1)
  number is 1
  print 1
  call countdown(0)

countdown(0)
  number is 0
  base case is true
  stop
```

30. Recursive Countdown Code Breakdown

```
function countdown(number) {
```

Create a function that receives a number.

```
  if (number < 1) {
    return;
  }
```

If the number is below , stop the function.

```
  console.log(number);
```

Print the current number.

```
  countdown(number - 1);
```

Call the same function again with a smaller number.

31. The Call Stack

The call stack is how JavaScript keeps track of function calls.

Simple meaning:

The call stack remembers which function calls are currently waiting to finish.

When a function calls another function, the new call goes on top of the stack.

Example:

```
countdown(3);
```

Conceptual stack:

```
countdown(3)
  countdown(2)
    countdown(1)
      countdown(0)
```

When `countdown(0)` stops, JavaScript returns back through the waiting calls.

32. Work Before the Recursive Call

This version prints before calling itself:

```
function countdown(number) {
  if (number < 1) return;

  console.log(number);
  countdown(number - 1);
}

countdown(3);
```

Output:

```
3
2
1
```

Why?

Because each number is printed before the next recursive call happens.

33. Work After the Recursive Call

This version prints after calling itself:

```
function countdown(number) {  
  if (number < 1) return;  
  
  countdown(number - 1);  
  console.log(number);  
}  
  
countdown(3);
```

Output:

```
1  
2  
3
```

Why?

Because JavaScript goes all the way down to the base case first.

Then it prints while the call stack resolves back upward.

34. Why Output Reverses

In this version:

```
countdown(number - 1);  
console.log(number);
```

JavaScript must finish the recursive call before it reaches `console.log(number)`.

So the deepest call finishes first.

That is why the output becomes:

```
1  
2  
3
```

35. Recursion Beginner Rule

When writing recursion, always ask:

1. What is the base case?
2. How does each recursive call move closer to the base case?
3. What should happen before the recursive call?
4. What should happen after the recursive call?

If the recursive call does not move closer to the base case, the function may never stop.

Part F: Divide and Conquer

36. What Is Divide and Conquer?

Divide and conquer is a problem-solving strategy.

Simple meaning:

Break a big problem into smaller problems, solve the smaller problems, then combine the answers.

Pattern:

```
Big problem
↓
Split into smaller problems
↓
Solve the smaller problems
↓
Combine the results
```

37. Why Divide and Conquer Is Useful

Some problems become easier when split into parts.

Examples:

- Binary search splits the search area in half.
- Merge sort splits the array into halves.
- Recursive algorithms often solve smaller versions of the same problem.

38. Binary Search as Divide and Conquer

Binary search divides the search problem.

Example:

[3, 8, 12, 18, 25, 40, 60]

Check the middle:

18

If the target is bigger than 18, ignore the left half:

[25, 40, 60]

The problem becomes smaller.

39. Merge Sort as Divide and Conquer

Merge sort divides the sorting problem.

Example:

[8, 3, 5, 1]

Split:

[8, 3] and [5, 1]

Split again:

[8] [3] [5] [1]

Then combine in sorted order.

Part G: Merging Two Sorted Arrays

40. Why Learn Merge First?

Before learning merge sort, you need to understand merging.

Merge sort depends on this smaller skill:

Take two sorted arrays and combine them into one sorted array.

Example:

```
merge([3, 8], [1, 5]);
```

Expected result:

```
[1, 3, 5, 8]
```

41. Important Rule

The merge function assumes both input arrays are already sorted.

Correct:

```
merge([3, 8], [1, 5]);
```

Both arrays are sorted.

Incorrect assumption:

```
merge([8, 3], [5, 1]);
```

These arrays are not sorted, so the merge logic will not work correctly.

42. Merge Function Code

```
function merge(left, right) {  
  const result = [];  
  let i = 0;
```

```
let j = 0;

while (i < left.length && j < right.length) {
  if (left[i] <= right[j]) {
    result.push(left[i]);
    i++;
  } else {
    result.push(right[j]);
    j++;
  }
}

return result.concat(left.slice(i)).concat(right.slice(j));
}

console.log(merge([3, 8], [1, 5])); // [1, 3, 5, 8]
```

43. Merge Code Breakdown

```
const result = [];
```

Create an empty array for the final sorted result.

```
let i = 0;
let j = 0;
```

Create two index variables.

- `i` tracks the current item in `left`
- `j` tracks the current item in `right`

```
while (i < left.length && j < right.length) {
```

Keep comparing while both arrays still have items left.

```
  if (left[i] <= right[j]) {
    result.push(left[i]);
    i++;
  }
```

If the current left item is smaller or equal, add it to `result` and move `i` forward.

```
else {  
    result.push(right[j]);  
    j++;  
}
```

Otherwise, add the current right item and move `j` forward.

```
return result.concat(left.slice(i)).concat(right.slice(j));
```

After one array is empty, add the remaining items from the other array.

44. Merge Dry Run

Input:

```
left = [3, 8];  
right = [1, 5];
```

Dry run:

Step	left[i]	right[j]	Smaller Value	Result
1	3	1	1	[1]
2	3	5	3	[1, 3]
3	8	5	5	[1, 3, 5]
4	8	no item	8	[1, 3, 5, 8]

Final result:

```
[1, 3, 5, 8]
```

45. Why `slice(i)` and `slice(j)` Are Used

At the end of the loop, one array may still have leftover items.

Example:

```
left = [8];  
right = [];
```

Since `left` is already sorted, the leftover items can be added directly.

```
left.slice(i)
```

means:

Get the remaining items from `left`, starting at index `i`.

```
right.slice(j)
```

means:

Get the remaining items from `right`, starting at index `j`.

Part H: Merge Sort

46. What Is Merge Sort?

Merge sort is a sorting algorithm.

Simple meaning:

Merge sort splits an array into smaller arrays, sorts them, then merges them back together.

Merge sort uses divide and conquer.

47. Merge Sort Main Steps

Merge sort follows this process:

1. Split the array into two halves.
2. Keep splitting until each array has one item or no items.
3. Merge the small arrays back together in sorted order.
4. Return the fully sorted array.

48. Why Single-Item Arrays Are Already Sorted

A single-item array is already sorted because there is nothing to compare.

Example:

```
[8]
```

This is already sorted.

Another example:

```
[3]
```

This is also already sorted.

That is why merge sort keeps splitting until arrays have length `1` or `0`.

49. Merge Sort Visual Example

Start:

```
[8, 3, 5, 1]
```

Split:

```
[8, 3] and [5, 1]
```

Split again:

```
[8] [3] [5] [1]
```

Merge sorted:

```
[3, 8] and [1, 5]
```

Merge again:

```
[1, 3, 5, 8]
```

Final sorted array:

```
[1, 3, 5, 8]
```

50. Merge Sort Code

```
function mergeSort(arr) {  
  if (arr.length <= 1) {  
    return arr;  
  }  
  
  const middle = Math.floor(arr.length / 2);  
  const left = arr.slice(0, middle);  
  const right = arr.slice(middle);  
  
  const sortedLeft = mergeSort(left);  
  const sortedRight = mergeSort(right);  
  
  return merge(sortedLeft, sortedRight);  
}  
  
function merge(left, right) {  
  const result = [];  
  let i = 0;  
  let j = 0;  
  
  while (i < left.length && j < right.length) {  
    if (left[i] <= right[j]) {  
      result.push(left[i]);  
      i++;  
    } else {  
      result.push(right[j]);  
      j++;  
    }  
  }  
  
  return result.concat(left.slice(i)).concat(right.slice(j));  
}  
  
console.log(mergeSort([8, 3, 5, 1])); // [1, 3, 5, 8]
```

51. Merge Sort Code Breakdown

```
function mergeSort(arr) {
```

Create a function that receives an array.

```
  if (arr.length <= 1) {  
    return arr;  
  }
```

This is the base case.

If the array has or item, it is already sorted.

```
  const middle = Math.floor(arr.length / 2);
```

Find the middle index.

```
  const left = arr.slice(0, middle);
```

Create the left half.

```
  const right = arr.slice(middle);
```

Create the right half.

```
  const sortedLeft = mergeSort(left);
```

Sort the left half using the same function.

```
  const sortedRight = mergeSort(right);
```

Sort the right half using the same function.

```
  return merge(sortedLeft, sortedRight);
```

Merge the sorted halves into one sorted array.

52. Merge Sort Dry Run

Call:

```
mergeSort([8, 3, 5, 1]);
```

Step 1: Split the original array.

```
[8, 3, 5, 1]
↓
[8, 3] and [5, 1]
```

Step 2: Split again.

```
[8, 3] → [8] and [3]
[5, 1] → [5] and [1]
```

Step 3: Base case reached.

```
[8], [3], [5], [1]
```

Each has one item, so each is already sorted.

Step 4: Merge sorted pieces.

```
merge([8], [3]) → [3, 8]
merge([5], [1]) → [1, 5]
```

Step 5: Merge final halves.

```
merge([3, 8], [1, 5]) → [1, 3, 5, 8]
```

Final result:

```
[1, 3, 5, 8]
```

53. Merge Sort Uses Recursion

Merge sort is recursive because `mergeSort()` calls itself.

```
const sortedLeft = mergeSort(left);
const sortedRight = mergeSort(right);
```

Each call receives a smaller array.

Eventually, each array becomes length `1` or `0`.

That reaches the base case:

```
if (arr.length <= 1) {
  return arr;
}
```

54. Merge Sort Uses Divide and Conquer

Merge sort uses divide and conquer like this:

Divide and Conquer Step	Merge Sort Version
Divide	Split the array into halves.
Conquer	Recursively sort each half.
Combine	Merge the sorted halves.

55. Merge Sort Time Complexity

Merge sort is usually **$O(n \log n)$** time.

Simple meaning:

Merge sort splits the array into levels, and each level does merging work.

Why `log n`?

Because the array keeps getting cut in half.

Why `n`?

Because each level must process all items during merging.

Together:

```
O(n log n)
```

56. Merge Sort Space Complexity

Merge sort usually uses **O(n)** extra space.

Why?

Because it creates new arrays when splitting and merging:

```
const left = arr.slice(0, middle);  
const right = arr.slice(middle);  
const result = [];
```

The extra memory grows with the size of the input.

57. Why Merge Sort Is Useful

Merge sort is useful because:

- it is efficient for large arrays
- it follows a clear divide-and-conquer structure
- it has predictable $O(n \log n)$ time complexity
- it is a strong example of recursion

But it is not always the simplest choice for small beginner tasks.

For small arrays, built-in sorting is usually enough:

```
const numbers = [8, 3, 5, 1];  
numbers.sort((a, b) => a - b);  
console.log(numbers); // [1, 3, 5, 8]
```

However, learning merge sort helps you understand algorithms more deeply.

Part I: Time and Space Complexity Review

58. Complexity Terms in Sprint 22

Complexity	Simple Meaning	Sprint 22 Example
$O(1)$	Constant work or memory.	Iterative binary search extra space.
$O(n)$	Work grows with input size.	Linear search time.
$O(\log n)$	Problem is repeatedly cut down.	Binary search time.
$O(n \log n)$	Repeated splitting plus full processing at each level.	Merge sort time.
$O(n)$ space	Extra memory grows with input size.	Merge sort extra arrays.

59. Search Complexity Comparison

Algorithm	Time Complexity	Space Complexity	Requirement
Linear search	$O(n)$	$O(1)$	No sorting required.
Binary search	$O(\log n)$	$O(1)$	Array must be sorted.

60. Sort Complexity Summary

Algorithm	Time Complexity	Space Complexity	Main Idea
Merge sort	$O(n \log n)$	$O(n)$	Split, sort, merge.

Part J: Common Beginner Mistakes

61. Mistake: Using Binary Search on an Unsorted Array

Wrong:

```
binarySearch([40, 3, 25, 8, 60], 25);
```

Problem:

The array is not sorted.

Correction:

Use linear search or sort the array first.

```
linearSearch([40, 3, 25, 8, 60], 25);
```

62. Mistake: Forgetting `return -1`

Wrong:

```
function linearSearch(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
}
```

Problem:

If the target is not found, the function returns `undefined`.

Better:

```
function linearSearch(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
  
  return -1;  
}
```

63. Mistake: Using `low < high` Incorrectly

Risky:

```
while (low < high) {
```

This may skip the final possible item.

Better for standard binary search:

```
while (low <= high) {
```

64. Mistake: Forgetting the Recursion Base Case

Wrong:

```
function countdown(number) {  
  console.log(number);  
  countdown(number - 1);  
}
```

Problem:

The function never stops.

Better:

```
function countdown(number) {  
  if (number < 1) return;  
  
  console.log(number);  
  countdown(number - 1);  
}
```

65. Mistake: Recursive Call Does Not Move Toward the Base Case

Wrong:

```
function countdown(number) {  
  if (number < 1) return;  
  
  console.log(number);  
  countdown(number);  
}
```

Problem:

The number never changes.

The function keeps calling itself with the same value.

Better:

```
countdown(number - 1);
```

66. Mistake: Thinking Recursion Is Magic

Recursion is not magic.

It is repeated function calls.

Each call has its own value for the parameter.

Example:

```
countdown(3)
countdown(2)
countdown(1)
countdown(0)
```

67. Mistake: Trying Merge Sort Before Understanding Merge

Merge sort depends on merge.

Before writing merge sort, you should be able to write this:

```
merge([3, 8], [1, 5]); // [1, 3, 5, 8]
```

If merging two sorted arrays is unclear, merge sort will feel confusing.

Part K: Minimum Passing Standard for Sprint 22

68. You Understand Linear Search If You Can Explain This

```
function linearSearch(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) {
      return i;
    }
  }
}
```

```
    }  
  }  
  
  return -1;  
}
```

You should be able to say:

Start at index 0.
Check each item.
If the current item matches the target, return its index.
If the loop ends with no match, return -1.

69. You Understand Binary Search If You Can Explain This

```
function binarySearch(arr, target) {  
  let low = 0;  
  let high = arr.length - 1;  
  
  while (low <= high) {  
    const mid = Math.floor((low + high) / 2);  
  
    if (arr[mid] === target) return mid;  
  
    if (arr[mid] < target) {  
      low = mid + 1;  
    } else {  
      high = mid - 1;  
    }  
  }  
  
  return -1;  
}
```

You should be able to say:

low starts at the first index.
high starts at the last index.
mid checks the middle index.
If the middle value is the target, return the index.
If the middle value is too small, ignore the left half.

If the middle value is too large, ignore the right half.
If the loop ends, the target was not found, so return -1.

70. You Understand Recursion If You Can Explain This

```
function countdown(number) {  
  if (number < 1) return;  
  
  console.log(number);  
  countdown(number - 1);  
}
```

You should be able to say:

The base case is `number < 1`.
The function prints the current number.
Then it calls itself with `number - 1`.
Each call moves closer to the base case.
The function stops when number becomes 0.

71. You Understand Merge Sort If You Can Explain This

```
function mergeSort(arr) {  
  if (arr.length <= 1) {  
    return arr;  
  }  
  
  const middle = Math.floor(arr.length / 2);  
  const left = arr.slice(0, middle);  
  const right = arr.slice(middle);  
  
  const sortedLeft = mergeSort(left);  
  const sortedRight = mergeSort(right);  
  
  return merge(sortedLeft, sortedRight);  
}
```

You should be able to say:

If the array has 0 or 1 item, return it because it is already sorted.
Otherwise, split the array into left and right halves.
Sort each half using `mergeSort` again.

Merge the sorted halves together.
Return the final sorted array.

Part L: Practice Tasks

72. Practice Task 1: Write Linear Search

Write a function named `linearSearch`.

It should:

- receive an array and a target
- return the index if the target exists
- return `-1` if the target does not exist

Test it with:

```
console.log(linearSearch([5, 10, 15], 10)); // 1  
console.log(linearSearch([5, 10, 15], 99)); // -1
```

73. Practice Task 2: Dry-Run Linear Search

Dry-run this call:

```
linearSearch([7, 14, 21, 28], 21);
```

Write each check:

```
7 === 21? no  
14 === 21? no  
21 === 21? yes  
return 2
```

74. Practice Task 3: Write Binary Search

Write a function named `binarySearch`.

It should:

- receive a sorted array and a target
- use `low`, `high`, and `mid`
- return the index if found
- return `-1` if not found

Test it with:

```
console.log(binarySearch([2, 4, 6, 8, 10, 12], 10)); // 4
console.log(binarySearch([2, 4, 6, 8, 10, 12], 99)); // -1
```

75. Practice Task 4: Dry-Run Binary Search

Dry-run:

```
binarySearch([2, 4, 6, 8, 10, 12], 10);
```

Expected dry run:

Step	low	high	mid	arr[mid]	Decision
1	0	5	2	6	10 is bigger, search right half.
2	3	5	4	10	Found. Return <code>4</code> .

76. Practice Task 5: Write Recursive Countdown

Write this from memory:

```
function countdown(number) {
  if (number < 1) return;

  console.log(number);
  countdown(number - 1);
}

countdown(3);
```

Expected output:

```
3  
2  
1
```

77. Practice Task 6: Reverse the Recursive Output

Move `console.log(number)` after the recursive call:

```
function countdown(number) {  
  if (number < 1) return;  
  
  countdown(number - 1);  
  console.log(number);  
}  
  
countdown(3);
```

Expected output:

```
1  
2  
3
```

Explain why the output changed.

78. Practice Task 7: Write Merge

Write a function named `merge`.

It should combine two sorted arrays into one sorted array.

Test it with:

```
console.log(merge([3, 8], [1, 5])); // [1, 3, 5, 8]  
console.log(merge([2, 6], [1, 4, 9])); // [1, 2, 4, 6, 9]
```

79. Practice Task 8: Write Merge Sort

Write `mergeSort()` using your `merge()` function.

Test it with:

```
console.log(mergeSort([8, 3, 5, 1])); // [1, 3, 5, 8]
console.log(mergeSort([10, -2, 7, 3])); // [-2, 3, 7, 10]
```

Part M: Sprint 22 Review Checklist

80. Concept Checklist

You are ready to move on from Sprint 22 when you can answer these questions:

- What is linear search?
- What does linear search return when the target is not found?
- Why is linear search $O(n)$?
- What is binary search?
- Why must binary search use sorted data?
- What do `low`, `high`, and `mid` mean?
- Why does binary search use `Math.floor()`?
- Why is binary search $O(\log n)$?
- What is recursion?
- What is a base case?
- What happens if recursion has no base case?
- What is the call stack?
- Why does moving `console.log()` after the recursive call reverse the output?
- What is divide and conquer?
- How does binary search use divide and conquer?
- How does merge sort use divide and conquer?
- What does `merge()` do?
- Why must `merge()` receive sorted arrays?
- What does `mergeSort()` do?
- Why is merge sort $O(n \log n)$?
- Why does merge sort use $O(n)$ extra space?

81. Code Checklist

You should be able to write these from memory:

- `linearSearch(arr, target)`
- `binarySearch(arr, target)`
- recursive `countdown(number)`
- `merge(left, right)`
- `mergeSort(arr)`

82. Dry-Run Checklist

You should be able to dry-run:

- linear search on a small array
 - binary search with `low`, `high`, and `mid`
 - recursive countdown
 - merging two sorted arrays
 - merge sort splitting and merging
-

Final Sprint 22 Summary

Sprint 22 teaches you how to search and sort data using smarter algorithms.

The simplest search is linear search. It checks each item one by one and works on unsorted data, but it is $O(n)$.

Binary search is faster because it cuts the search area in half each time. It is $O(\log n)$, but it only works on sorted data.

Recursion is when a function calls itself. Every recursive function needs a base case so it can stop.

Divide and conquer means splitting a big problem into smaller problems, solving those smaller problems, and combining the answers.

Merge sort uses recursion and divide and conquer. It splits an array into halves, sorts the halves, and merges them back together. Its time complexity is $O(n \log n)$, and its space complexity is $O(n)$.

Final sentence:

Sprint 22 teaches you how to search and sort data more intelligently by using loops, recursion, and divide-and-conquer thinking.